Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 21 · Part E: Machine Learning for Signals Level 3

Track: Feature Engineering, Envelope Analysis, Classical Classifiers, Honest Validation, Fault Diagnosis

Prerequisites: Problems 12, 13, 18 · **Estimated effort:** 14–18 hours

Features and Classical Machine Learning: Physics First, Classifier Second

A good feature is a piece of signal processing that makes the classes separable. Extract time-domain, spectral and envelope-spectrum features from vibration data, diagnose bearing faults, compare simple classifiers, and learn the most expensive mistake in applied machine learning: reporting accuracy from a validation split that shares recordings with the training set.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/21_features_and_classical_ml/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random seeds, (ii) a short written report (`report.pdf` or `report.md`) containing the requested analysis sections,

and (iii) all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

Machine learning on signals is two jobs: turning a waveform into numbers that carry the relevant physics, and learning a decision rule from those numbers. For decades the first job was done by hand, from the Fourier transform and statistics of problems 07 and 12, and it still beats deep learning when data are few or when the physics is known. A failing rolling-element bearing, for example, produces impacts at a characteristic repetition rate that is a known multiple of the shaft speed; the envelope spectrum shows it as a line even when the raw spectrum looks like noise.

The second job hides a trap. Real recordings are long, so they are cut into segments; segments from the same recording share the machine's speed, gain and resonance, and a classifier can memorise those nuisance values instead of learning the fault. Splitting the segments randomly into training and test sets then gives a flattering accuracy that vanishes when the system meets a new machine. This problem builds the pipeline, then shows you this trap and its remedy with a controlled experiment.

Theoretical Framework

1. Features of a Segment $x[n]$, $n = 0..N-1$

Time domain: mean, RMS, peak, crest factor $x_{\mathrm{pk}}/x_{\mathrm{rms}}$, skewness, kurtosis $\mathbb{E}\{(x-\bar{x})^4\}/\sigma^4$ (equal to 3 for Gaussian noise; impulsive faults raise it), zero-crossing rate. Frequency domain from the PSD $S[k]$: spectral centroid $\sum f_k S_k / \sum S_k$ (for white noise $f_s/4$), bandwidth, band powers, dominant frequency, spectral entropy. Envelope domain: band-pass around the structural resonance, take the analytic-signal magnitude $|x + \mathrm{j}\mathcal{H}\{x\}|$, remove its mean and compute its spectrum; a fault appears as lines at the fault frequency and its harmonics.

2. Bearing Fault Frequencies

For the drive-end bearing of the CWRU rig the fault frequencies are fixed multiples of the shaft frequency f_r: inner race $5.415 f_r$, outer race $3.585 f_r$, ball spin $4.714 f_r$ (the ball signature appears at twice that), cage $0.398 f_r$. The synthetic generator uses the same multiples.

3. Classifiers and Metrics

Logistic regression (linear boundary, probabilistic), RBF-kernel SVM (nonlinear, needs scaled features), random forest (bagged trees, robust, gives importances). Report accuracy, precision, recall, F1 and the confusion matrix; for imbalanced data use the precision–recall curve and average precision. Always compare with the trivial baseline (majority class or random).

4. Validation and Leakage

k-fold cross-validation estimates generalisation if folds are independent. Signal segments cut from one recording are *not* independent; use *group* cross-validation, splitting by recording (or machine, or day) so that no group appears on both sides. The gap between the random-split and group-split scores measures your leakage.

5. Decision Thresholds

A classifier's score becomes a decision at a threshold; choose it from the precision–recall or ROC curve to meet an operational requirement (for example recall ≥ 0.95 at the smallest false-alarm rate), not at the default 0.5.

Data and Simulation Specification

Dataset	`sp_common.synth.bearing_dataset(n_recordings_per_class=8, segments_per_recording=10, segment_s=0.5, fs=12000)`: 4 classes (none, inner, outer, ball), 320 segments of 6000 samples; per-recording nuisance variation in speed (28.5–30.5 Hz), severity, resonance (2.8–3.6 kHz), gain and noise; the `groups` vector gives recording ids.
Envelope analysis demo	`sp_common.synth.bearing_vibration(fault=...)`, 10 s records, shaft frequency 29.95 Hz.
Robustness	Added white noise to give input SNR from +10 to −10 dB; train on shaft speeds below 29.5 Hz and test above.
Real data (choose one)	(i) CWRU Bearing Data Center, 12 kHz drive-end files for normal, inner-race, outer-race and ball faults at a fixed load: segment into 0.5 s pieces and repeat the experiment. (ii) HTRU2 pulsar candidates (UCI repository): 17 898 candidates, 8 pre-computed features, 1 639 real pulsars; a strongly imbalanced problem.

Your Task

1. **Feature library (Stage 2).** Implement functions for time-domain, spectral and envelope features. Test them on signals with known answers (sine, Gaussian noise, square wave: RMS, kurtosis, centroid, crest factor) and compute the feature matrix for the whole dataset. Show a 2-D PCA projection coloured by class and by recording.
2. **Envelope analysis.** For one 10-s record of each fault type band-pass around the resonance, plot the envelope spectrum and mark the predicted fault frequency and harmonics. Compare detectability with the ordinary spectrum. Implement a simple kurtogram (kurtosis of the envelope over centre frequency and bandwidth) to find the best demodulation band automatically.
3. **Baselines.** Build scikit-learn pipelines (scaler plus logistic regression, SVM and random forest) and evaluate with (a) random stratified 5-fold CV and (b) `GroupKFold` by recording. Report the accuracy of the trivial baseline, both CV scores for each model and the optimism gap. Explain mechanistically why the gap occurs, using the PCA plot of Task 1.

4. **Feature ablation and importance.** Repeat the group-CV evaluation with time-only, spectral-only, envelope-only and all features; compute permutation importance for the random forest. Identify which fault class is hardest and why (use the confusion matrix).
5. **Robustness and domain shift.** Plot group-CV accuracy against input SNR; train on low shaft speeds and test on high ones; explain which features are speed-dependent and how you would make them speed-invariant (order tracking, normalising by shaft frequency).
6. **Learning curves and calibration.** Plot learning curves (accuracy versus number of training recordings). Check probability calibration with a reliability diagram and calibrate the best model. Choose a decision threshold that gives recall ≥ 0.95 for "fault versus no fault" and report the false-alarm rate it costs.
7. **Real data.** Repeat the pipeline on CWRU, *splitting by file*, or on HTRU2 with class weights, a precision–recall curve and a threshold chosen for recall ≥ 0.95. Compare with the synthetic results and discuss what differs.

Self-check targets

Four balanced classes give a trivial accuracy of 0.25. Gaussian noise has kurtosis 3 and spectral centroid $f_s/4$; a unit-amplitude sine has crest factor $\sqrt{2}$. For the outer-race fault the envelope spectrum shows a line at $3.585 \times 29.95 = 107.4$ Hz (inner race 162.2 Hz). Random-split CV is expected to score higher than group CV; report both and the difference.

Aerospace Application Link

Reaction-wheel bearing degradation is a recurring cause of satellite anomalies, and wheel speed varies continuously. Describe how you would deploy your classifier on a satellite: which features survive speed changes, what the onboard processor budget is for computing them (operations per 0.5 s segment), what recall and false-alarm rate you require given that a false alarm triggers a safe-mode event and a miss risks losing the spacecraft, and how many labelled failures you can realistically have (which limits the model).

Deliverables

D1 **Feature library:** tested functions, the feature matrix and the PCA plot.

D2 **Envelope analysis:** envelope spectra for the four classes, the kurtogram.

D3 **Baseline models:** random versus group CV table for the three models, the trivial baseline, mechanistic explanation of the gap.

D4 **Ablation and robustness:** feature-group study, permutation importance, SNR and speed-shift experiments.

D5 **Calibration, threshold and real data:** reliability diagram, operating point, CWRU or HTRU2 results.

D6 **Memo (300 words):** "Can we trust the 99% accuracy?" Explain leakage and give your honest performance estimate with uncertainty. Cite at least two external sources.

Bonus (a) Implement order tracking to remove the speed dependence. (b) Add cepstral features and compare. (c) Use a leave-one-machine-out evaluation on real CWRU loads (0–3 hp) to measure cross-load generalisation.

Learning Resources

Features and Machine Condition Monitoring

- Randall, R.B. *Vibration-Based Condition Monitoring.*
- Smith, W.A., Randall, R.B. (2015). "Rolling element bearing diagnostics using the Case Western Reserve University data: a benchmark study." Mech. Syst. Signal Process. 64–65, 100.
- Antoni, J. (2007). "Fast computation of the kurtogram for the detection of transient faults." Mech. Syst. Signal Process. 21, 108.

Machine Learning

- Hastie, T., Tibshirani, R., Friedman, J. *The Elements of Statistical Learning*, chapters on model assessment, SVM and random forests.
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow.*
- Lyon, R.J. et al. (2016). "Fifty years of pulsar candidate selection: from simple filters to a new principled real-time classification approach." Mon. Not. R. Astron. Soc. 459, 1104.

Data and Tools

- CWRU Bearing Data Center: `https://engineering.case.edu/bearingdatacenter`; HTRU2 at the UCI Machine Learning Repository: `https://archive.ics.uci.edu`
- scikit-learn (`GroupKFold`, `Pipeline`, `permutation_importance`): `https://scikit-learn.org/stable/`

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.